D.1 Limiting Behaviors

D.1.1 Small β_t

When β_t is small, we have the following theorem showing that $\ell_t(\theta)$ in Eqn 10 is **approximately the cross-entropy loss** at position t, up to a constant.

Theorem 1. *For a given θ, as $\beta_t \to 0$, we have*

$$\ell_t(\theta) - C(\beta_t) = \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[-\mathbb{1}_{\{t\le|\boldsymbol{y}|\}} \cdot \log \pi_\theta(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t})\right] + O(\beta_t), \tag{12}$$

where

$$C(\beta_t) := \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\left(\frac{2}{\beta_t}\log 2 + \log \pi_{\text{aligned}}(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t})\right)\right], \tag{13}$$

which is a bias term that is constant with respect to θ.

Proof. Recall that $\ell_t(\theta) := \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\,\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\frac{2}{\beta_t}S(\beta_t\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t))$ and $S(z) := \log(1+e^z)$ is the softplus function [41]. By Taylor expansion, it holds for all z that $S(\beta_t z) = S(0) + S'(0)\beta_t z + S''(\xi\beta_t z)\beta_t^2 z^2$ for some $\xi \in (0,1)$ depending on z. Since $S(0) = \log 2$, $S'(0) = \frac{1}{2}$ and $S''(x) \in [0, 1/4]$ for all x, we have $|S(\beta_t z) - (\log 2 + \frac{1}{2}\beta_t z)| \le \frac{1}{4}\beta_t^2 z^2$. Dividing both sides by $\beta_t/2$, we get

$$\left|\frac{2}{\beta_t}S(\beta_t z) - \left(\frac{2}{\beta_t}\log 2 + z\right)\right| \le \frac{1}{2}\beta_t z^2. \tag{14}$$

Then for our token-wise loss $\ell_t(\theta)$, we have

$$\begin{aligned}\left|\ell_t(\theta) - \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\left(\frac{2}{\beta_t}\log 2 + \Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t)\right)\right]\right| &\le \frac{1}{2}\beta_t \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\Delta_t^2(\boldsymbol{x},\boldsymbol{y}_{<t},y_t)\\ &= O(\beta_t).\end{aligned}$$

Recall that $\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t) := \log \pi_{\text{aligned}}\big(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}\big) - \log \pi_\theta\big(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}\big)$ as per Eqn 11. We can thus have

$$\begin{aligned}\ell_t(\theta) - \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\left(\frac{2}{\beta_t}\log 2 + \log \pi_{\text{aligned}}(y_t|\boldsymbol{x},\boldsymbol{y}_{<t})\right)\right]&\\ = -\mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\log \pi_\theta(y_t|\boldsymbol{x},\boldsymbol{y}_{<t})\right] + O(\beta_t).&\end{aligned} \tag{15}$$

Noting that $C(\beta_t) := \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\left(\frac{2}{\beta_t}\log 2 + \log \pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})\right)\right]$, we can complete the proof. □

D.1.2 Large β_t

When β_t is large, we have the following theorem showing that $\ell_t(\theta)$ can be approximated by $\tilde{\ell}_t(\theta) := \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\max\{\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t),0\}\right]$.

Theorem 2. *For a given θ, as $\beta_t \to +\infty$, we have*

$$\ell_t(\theta) = 2\tilde{\ell}_t(\theta) + O(\beta_t^{-1}),$$

where

$$\tilde{\ell}_t(\theta) := \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}}\max\{\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t),0\}\right].$$

Proof. First, we note the following identity.

$$S(z) = \log(1+e^z) = \max\{z,0\} + \log((1+e^z)e^{-\max\{z,0\}}) = \max\{z,0\} + \log(1+e^{-|z|}). \tag{16}$$

This implies that

$$\left|\frac{2}{\beta_t}S(\beta_t z) - 2\cdot\max\{z,0\}\right| = \frac{2}{\beta_t}\log(1+e^{-\beta_t|z|}) \le \frac{2}{\beta_t}e^{-\beta_t|z|}. \tag{17}$$

Then for our token-wise loss $\ell_t(\theta)$, we have

$$\left|\ell_t(\theta) - 2 \cdot \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}} \max\{\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t),0\}\right]\right| \le \frac{1}{\beta_t} \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D}\left[\mathbb{1}_{\{t\le|\boldsymbol{y}|\}} e^{-\beta_t|\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t)|}\right] \tag{18}$$

Noting that the RHS is $O(\beta_t^{-1})$ completes the proof. □

Next, we show that $\tilde{\ell}_t(\theta)$ is minimized to 0 if and only if $\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) = \pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$.

Theorem 3. *The minimum value of $\tilde{\ell}_t(\theta)$ is 0. If $\Pr_{(\boldsymbol{x},\boldsymbol{y})\sim D}[y_t = c \mid \boldsymbol{x},\boldsymbol{y}_{<t}] > 0$ for all $(\boldsymbol{x},\boldsymbol{y})$ in the support of D and all c in the vocabulary, then $\tilde{\ell}_t(\theta) = 0$ if and only if $\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) = \pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$.*

Proof. It is obvious that $\tilde{\ell}_t(\theta) \ge 0$, and $\tilde{\ell}_t(\theta) = 0$ can be attained when θ stays the same as the parameter for π_{aligned}. It is obvious that $\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) = \pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$ implies $\tilde{\ell}_t(\theta) = 0$. Conversely, suppose $\tilde{\ell}_t(\theta) = 0$. Then $\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t) \le 0$ for all $(\boldsymbol{x},\boldsymbol{y})$ in the support of D. By definition of $\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t)$, this implies $\pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) \le \pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$. Summing over all y_t in the vocabulary, we get $1 = \sum_{y_t} \pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) \le \sum_{y_t} \pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) = 1$. So all the inequalities must be equalities, and we get $\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) = \pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$. □

This corresponds to the intuition that large β_t places emphasis on matching the generative distribution of fine-tuned model to the initial aligned model.

D.2 Gradients of The Constrained Fine-tuning Objective

The gradient of $\ell_t(\theta)$ on a data point $(\boldsymbol{x},\boldsymbol{y})$ with $|\boldsymbol{y}| \ge t$ is derived as:

$$\begin{aligned}\nabla\left(\frac{2}{\beta_t}S(\beta_t\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t))\right) &= 2S'(\beta_t\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t))(-\nabla\log\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})) \\ &= 2\sigma(\beta_t\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t))(-\nabla\log\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})).\end{aligned} \tag{19}$$

Note that the gradient of the cross-entropy loss is $-\nabla\log\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$. Therefore, compared to vanilla cross-entropy loss, our fine-tuning objective essentially applies an additional adaptive weight $w_t := 2\cdot\sigma(\beta_t\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t))$ for the token-wise gradient term of cross-entropy. The weight w_t decreases as $\Delta_t(\boldsymbol{x},\boldsymbol{y}_{<t},y_t) := \log\pi_{\text{aligned}}(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t}) - \log\pi_\theta(y_t \mid \boldsymbol{x},\boldsymbol{y}_{<t})$ decreases. This difference in log probabilities essentially characterizes the deviation between the fine-tuned model π_θ and the initially aligned model π_{aligned} at the token position t (see also Theorem 3). Taking together, we can see that the fine-tuning objective will adaptively diminish the weight of those token positions where the deviation of the distribution approaches a certain threshold (controlled by β_t), thereby constraining it from further deviation (by diminishing the gradient at this position). Note that, at the beginning of the fine-tuning when π_θ is initialized as π_{aligned}, the weight $w_t = 1$, and the gradient of the loss is equivalent to that of standard cross-entropy loss.

D.3 Interpreting Eqn 3 from A Reinforcement Learning Perspective

We note that our loss function in Eqn 3 can also be interpreted from a reinforcement learning perspective if we **cast fine-tuning as a KL-constrained reinforcement learning problem** rather than a standard supervised fine-tuning problem. Specifically, we can follow a similar trick as DPO [16] to derive a unified loss, but taking a different approach to reward modeling. We, instead, formulate our problem setting like Mudgal et al. [40], where we optimize at the token level (where tokens are actions), rather than DPO which uses a sequence-level optimization (corresponding more to a bandit setting where entire sequences are actions).

D.3.1 A Token-wise Reinforcement Learning (RL) Formulation.

We first introduce the following token-wise RL formulation for LM alignment, which is adapted from Mudgal et al. [40]. In Appendix D.3.2, we show how a fine-tuning task can be cast into this token-wise RL problem, and Eqn 3 is essentially a surrogate learning objective of this RL problem.

Reward Function. For a pair of an input and a model response $(\boldsymbol{x}, \boldsymbol{y})$, we cast custom fine-tuning as a problem of further optimizing an already aligned model for a new custom reward function $r([\boldsymbol{x}, \boldsymbol{y}])$. Here we use $[\boldsymbol{x}, \boldsymbol{y}]$ to denote a concatenation of the two sequences and we use this concatenation to denote a state, and the reward function is defined on this state. Following Mudgal et al. [40], we can decompose it to a token-wise reward $R([\boldsymbol{x}, \boldsymbol{y}_{<t}])$ defined on the intermediate state $[\boldsymbol{x}, \boldsymbol{y}_{<t}]$:

$$R([\boldsymbol{x}, \boldsymbol{y}_{<t}]) = \begin{cases} 0, & y_{t-1} \neq EOS \\ r([\boldsymbol{x}, \boldsymbol{y}_{<t}]), & y_{t-1} = EOS \end{cases}, \tag{20}$$

where EOS is the end of the sequence token. The reward is nonzero only if the decoding is complete. We note that, by $r([\boldsymbol{x}, \boldsymbol{y}_{<t}])$ and $R([\boldsymbol{x}, \boldsymbol{y}_{<t}])$, we mean the function r and R are applied on the concatenation of $\boldsymbol{x}$ and $\boldsymbol{y}_{<t}$. Similarly, in the following, we will also use notations such as $R([\boldsymbol{x}, \boldsymbol{y}_{<t}, \boldsymbol{z}_{<\tau}])$ and $R([\boldsymbol{x}, \boldsymbol{y}_{<t}, z])$ to denote that we apply R on the concatenation between $[\boldsymbol{x}, \boldsymbol{y}_{<t}]$ and a followup sequence $\boldsymbol{z}_{<\tau}$ or just a single token z. These concatenations all represent some states of the generation.

Value Function. At an intermediate state $[\boldsymbol{x}, \boldsymbol{y}_{<t}]$, the value function of a policy π defined on this reward function can be written as:

$$V_{\pi}([\boldsymbol{x}, \boldsymbol{y}_{<t}]) := \mathop{\mathbb{E}}_{\boldsymbol{z} \sim \pi(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})} \left\{ \sum_{\tau \geq 1} R\Big([\boldsymbol{x}, \boldsymbol{y}_{<t}, \boldsymbol{z}_{<\tau}]\Big) \right\} \tag{21}$$

Here $\boldsymbol{z}$ is a sequence, and $\boldsymbol{z}_{<\tau}$ is empty when $\tau = 1$ as we index tokens in a sequence starting from the index number 1. Also, in the following formulations, we will have multiple different notations π_{θ}, π_{aligned}, π^* to denote different policies instances, so we will use $V_{\pi_{\theta}}$, $V_{\pi_{\text{aligned}}}$, V_{π^*} to differently denote their value functions respectively.

Advantage Function. When the custom fine-tuning is modeled by the reward function R, we define an expected advantage function of a fine-tuned model π_{θ} w.r.t. the initially aligned model π_{aligned}:

$$\hat{A}_{\pi_{\theta}}([\boldsymbol{x}, \boldsymbol{y}_{<t}]) := \mathop{\mathbb{E}}_{z \sim \pi_{\theta}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})} \left\{ V_{\pi_{\text{aligned}}}\Big([\boldsymbol{x}, \boldsymbol{y}_{<t}, z]\Big) - V_{\pi_{\text{aligned}}}\Big([\boldsymbol{x}, \boldsymbol{y}_{<t}]\Big) \right\}, \tag{22}$$

Here the advantage function is defined for non-terminal states $[\boldsymbol{x}, \boldsymbol{y}_{<t}]$ where y_{t-1} is not the ending token EOS, and z is a single token sampled by $z \sim \pi_{\theta}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})$. Note that the left-hand term could also be viewed as a Q-function with z being the action. In other words, a language model can be viewed as a fully observable Markov decision process (MDP) with state represented by the concatenation of the prompt and the partially decoded response tokens so far and action represented by the next token that is to be decoded.

The Reinforcement Learning Objective. Following Mudgal et al. [40], we adopt a token-wise RL learning objective:

$$\max_{\theta} \mathop{\mathbb{E}}_{(\boldsymbol{x}, \boldsymbol{y}) \sim D} \left\{ \sum_{t \geq 1} \Big[\tilde{A}_{\pi_{\theta}}([\boldsymbol{x}, \boldsymbol{y}_{<t}]) - \beta_t \cdot D_{\text{KL}}\big(\pi_{\theta}(\cdot\,|\boldsymbol{x}, \boldsymbol{y}_{<t}) \,\big\|\, \pi_{\text{aligned}}(\cdot\,|\boldsymbol{x}, \boldsymbol{y}_{<t})\big) \Big] \right\}, \tag{23}$$

where the advantage function is optimized at each token position t with a token-wise KL regularizer term $\beta_t \cdot D_{\text{KL}}\big(\pi_{\theta}(\cdot\,|\boldsymbol{x}, \boldsymbol{y}_{<t}) \,\big\|\, \pi_{\text{aligned}}(\cdot\,|\boldsymbol{x}, \boldsymbol{y}_{<t})\big)$ applied for each token position. The strength of regularization at each token position is controlled by β_t.

Closed Form of The Optimal Solution π^*. Omitting some intermediate steps for brevity, by Theorem 2.1 of Mudgal et al. [40], the optimal policy solution π^* of Eqn 23 is:

$$\pi^*(z|\boldsymbol{x}, \boldsymbol{y}_{<t}) = \frac{1}{Z(\boldsymbol{x}, \boldsymbol{y}_{<t})} \pi_{\text{aligned}}(z|\boldsymbol{x}, \boldsymbol{y}_{<t}) \cdot e^{V_{\pi^*}([\boldsymbol{x}, \boldsymbol{y}_{<t}, z])/\beta_t}, \tag{24}$$

where $Z(\boldsymbol{x}, \boldsymbol{y}_{<t})$ is the partition function, V_{π^*} is the value function of this optimal policy. We note that this conveniently allows us to re-arrange terms to represent the optimal value function—similar to the steps taken during the derivation of DPO [16]:

$$V_{\pi^*}([\boldsymbol{x}, \boldsymbol{y}_{<t}]) = \beta_t \cdot \log \frac{\pi^*(z|\boldsymbol{x}, \boldsymbol{y}_{<t})}{\pi_{\text{aligned}}(z|\boldsymbol{x}, \boldsymbol{y}_{<t})} + \beta_t \cdot \log Z(\boldsymbol{x}, \boldsymbol{y}_{<t}) \tag{25}$$

D.3.2 Casting Fine-tuning into a Reinforcement Learning Objective

In the setting of custom fine-tuning we consider in this work, the dataset D is in the form of $D := \{\boldsymbol{x}, \boldsymbol{y}\}$ with only inputs and example outputs, without preference pairs. Now, we show an alternative to the standard SFT objective (Eqn 1) for learning from this dataset: casting the optimization as a KL-regularized RL objective in Appendix D.3.1. We will show how our token-wise constrained fine-tuning objective in Eqn 3 can essentially be derived from this token-wise KL-regularized RL problem!

Intuitively, fine-tuning π_{aligned} further on custom data points $(\boldsymbol{x}, \boldsymbol{y})$ implicitly assumes that this fine-tuning data is as preferable or more preferable than whatever the model would currently output. To represent this implied preference that the custom example response $\boldsymbol{y}$ is better than the current responses from π_{aligned}, we can effectively leverage existing preference learning methods. Recall that, in preference optimization (with both positive and negative examples), the Bradley-Terry model [62] is used as a model of the underlying reward function. In our setup, we don't have pair-wise preference data, and we only have inputs and positive examples. However, we can define a boolean $T([\boldsymbol{x}, \boldsymbol{y}_{<t}], y_t)$ to encode the preference: in the fine-tuning task, y_t is a preferable token output following $[\boldsymbol{x}, \boldsymbol{y}_{<t}]$ compared to the responses from the initial aligned model π_{aligned}. So, given an expected random draw from the aligned policy and the draw from the fine-tuned optimal policy, we can define:

$$\mathbb{P}\left(T([\boldsymbol{x}, \boldsymbol{y}_{<t}], y_t)\right) = \sigma\left(V_{\pi^*}([\boldsymbol{x}, \boldsymbol{y}_{<t}, y_t]) - \mathop{\mathbb{E}}_{z \sim \pi_{\text{aligned}}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})} V_{\pi^*}([\boldsymbol{x}, \boldsymbol{y}_{<t}, z])\right) \tag{26}$$

Intuitively, this means that—conditioned on the context $[\boldsymbol{x}, \boldsymbol{y}_{\leq t}]$—if there is a continuation token y_t that is higher than the average reward of actions sampled from the initial aligned model π_{aligned}, it is likely to be an improved or preferred choice in the custom fine-tuning task. The higher the margin $V_{\pi^*}([\boldsymbol{x}, \boldsymbol{y}_{<t}, y_t]) - \mathbb{E}_{z \sim \pi_{\text{aligned}}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})} V_{\pi^*}([\boldsymbol{x}, \boldsymbol{y}_{<t}, z])$ is, the more likely it is an improvement.

With this function in mind, combined with Eqn 25, we can leverage a similar derivation to DPO [16] to arrive at a constrained fine-tuning objective that is only dependent on the current policy, but is regularized by the original aligned policy. We plug in the closed form of the optimal value function (Eqn 25) into the modeling in Eqn 26, obtaining:

$$\begin{aligned}
\mathbb{P}\left(T([\boldsymbol{x}, \boldsymbol{y}_{<t}], y_t)\right) &= \sigma\left(\beta_t \log \frac{\pi^*(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})}{\pi_{\text{aligned}}(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})} - \beta_t \mathop{\mathbb{E}}_{z \sim \pi_{\text{aligned}}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})} \log \frac{\pi^*(z|\boldsymbol{x}, \boldsymbol{y}_{<t})}{\pi_{\text{aligned}}(z|\boldsymbol{x}, \boldsymbol{y}_{<t})}\right) \\
&= \sigma\left(\beta_t \log \frac{\pi^*(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})}{\pi_{\text{aligned}}(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})} + \beta_t D_{\text{KL}}\Big(\pi^*(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t}) \Big\| \pi_{\text{aligned}}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})\Big)\right).
\end{aligned} \tag{27}$$

Thus, we don't need to explicitly learn the value function, instead it is implicitly encoded by the policy. Then the optimization objective can become:

$$\max_{\theta} \mathop{\mathbb{E}}_{(\boldsymbol{x}, \boldsymbol{y}) \sim D} \left\{ \sum_{t \geq 1} \frac{1}{\beta_t} \log \mathbb{P}_\theta\left(T([\boldsymbol{x}, \boldsymbol{y}_{<t}], y_t)\right) \right\}, \tag{28}$$

where:

$$\mathbb{P}_\theta\left(T([\boldsymbol{x}, \boldsymbol{y}_{<t}], y_t)\right) := \sigma\left(\beta_t \log \frac{\pi_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})}{\pi_{\text{aligned}}(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})} + \beta_t D_{\text{KL}}\Big(\pi^*(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t}) \Big\| \pi_{\text{aligned}}(\cdot|\boldsymbol{x}, \boldsymbol{y}_{<t})\Big)\right), \tag{29}$$